DAY 1 TRAINING REPORT

C Programming Fundamentals

Topics Covered

1. Introduction to C
2. Compilation Process
3. Identifiers
4. Keywords
5. Variables
6. Constants
7. Data Types
8. Modifiers in C
9. Type Casting

---

1. Introduction to C

Overview

C is a general-purpose programming language developed by Dennis Ritchie in 1972 at Bell Laboratories. It is widely used for system programming, embedded systems, operating systems, and application development.

Features of C

- Simple and efficient
- Structured programming language
- Fast execution speed
- Portable language

- Supports low-level programming

Applications of C

- Operating Systems
- Compilers
- Interpreters
- Databases
- Text Editors
- Network Drivers

Basic Structure of a C Program

```c
#include<stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

Practice Programs

Program 1 – Simple Output

```c
#include<stdio.h>

int main()
{
    printf("Student");
    return 0;
}
```

Output

```
Student
```

Program 2 – Escape Sequence

```c
#include<stdio.h>

int main()
{
    printf("HI\nHELLO");
    return 0;
}
```

Output

```
HI
HELLO
```

Program 3 – Double Quotes Inside String

```c
#include<stdio.h>

int main()
{
    printf("Slow and \"Steady\" wins the race");
    return 0;
}
```

Output

```
Slow and "Steady" wins the race
```

Program 4 – Single Quotes Inside String

```c
#include<stdio.h>

int main()
{
    printf("Learn 'C' language");
    return 0;
}
```

Output

```
Learn 'C' language
```

---

2. Compilation Process

Overview

The compilation process converts C source code into machine-level executable code.

Stages of Compilation

1. Preprocessing

2. Compilation

3. Assembly
4. Linking

Compilation Command

```
gcc filename.c
```

---

3. Identifiers

Overview

Identifiers are user-defined names given to variables, functions, arrays, and structures.

Rules for Identifiers

- Can contain alphabets, digits, and underscore (_)
- Must begin with an alphabet or underscore
- Cannot start with a number
- Spaces are not allowed
- Special symbols are not allowed
- Keywords cannot be used as identifiers

Example

```
int studentAge;
float total_marks;
```

---

4. Keywords

Overview

Keywords are reserved words in C that have predefined meanings.

Common Keywords

```
int
float
char
if
else
while
```

```
return
const
```

---

5. Variables

Overview

Variables are memory containers used to store data values.

Syntax

```
datatype variable_name;
```

Practice Programs

Integer Input and Output

```c
#include<stdio.h>

int main()
{
    int a;

    scanf("%d",&a);
    printf("%d",a);

    return 0;
}
```

Input

```
10
```

Output

```
10
```

Signed Integer Format

```c
#include<stdio.h>

int main()
{
    int b;
```

```
    scanf("%d",&b);
    printf("%+d",b);

    return 0;
}
```

Input

```
25
```

Output

```
+25
```

Character ASCII Value

```
#include<stdio.h>

int main()
{
    char ch;

    scanf("%c",&ch);
    printf("%d",ch);

    return 0;
}
```

Input

```
A
```

Output

```
65
```

---

6. Constants

Overview

Constants are fixed values whose values cannot be modified during program execution.

Types of Constants

- Integer Constant
- Floating Constant
- Character Constant

- String Constant

Syntax

```
const datatype variable_name = value;
```

Example Program

```
#include<stdio.h>

int main()
{
    const int a = 10;

    a = 12;

    printf("%d",a);

    return 0;
}
```

Output

```
Error: assignment of read-only variable 'a'
```

---

7. Data Types

Overview

Data types specify the type of data a variable can store.

Commonly Used Data Types

- int
- char
- float
- double
- short int
- long int
- long long int
- signed int
- unsigned int

Practice Programs

Short Integer

```c
#include<stdio.h>

int main()
{
    short int a = 5;

    printf("%hd",a);

    return 0;
}
```

Output

```
5
```

Long Long Integer

```c
#include<stdio.h>

int main()
{
    long long int n;

    scanf("%lld",&n);
    printf("%lld",n);

    return 0;
}
```

Input

```
9876543210123
```

Output

```
9876543210123
```

Float Input and Output

```c
#include<stdio.h>

int main()
{
    float f;
```

```
    scanf("%f",&f);
    printf("%f",f);

    return 0;
}
```

Input

```
12.45
```

Output

```
12.450000
```

Float Precision Control

```
#include<stdio.h>

int main()
{
    float f;

    scanf("%f",&f);
    printf("%.3f",f);

    return 0;
}
```

Input

```
12.45678
```

Output

```
12.457
```

Double Precision

```
#include<stdio.h>

int main()
{
    double n;

    scanf("%lf",&n);
    printf("%.15lf",n);
```

```
    return 0;
}
```

Input

```
12.123456789123456
```

Output

```
12.123456789123456
```

---

8. Modifiers in C

Overview

Modifiers are used to change the range, memory size, and behavior of data types.

Categories of Modifiers

1. Sign Modifiers
2. Size Modifiers

---

signed Modifier

Syntax

```
signed int variable_name;
```

Example

```
#include<stdio.h>

int main()
{
    signed int a;

    scanf("%d",&a);
    printf("%d",a);

    return 0;
}
```

Input

```
-25
```

Output

```
-25
```

---

unsigned Modifier

Syntax

```
unsigned int variable_name;
```

Example

```
#include<stdio.h>

int main()
{
    unsigned int a;

    scanf("%u",&a);
    printf("%u",a);

    return 0;
}
```

Input

```
50
```

Output

```
50
```

---

short Modifier

```
#include<stdio.h>

int main()
{
    short int a = 32000;

    printf("%hd",a);

    return 0;
}
```

Output

```
32000
```

---

long Modifier

```c
#include<stdio.h>

int main()
{
    long int population = 987654321;

    printf("%ld",population);

    return 0;
}
```

Output

```
987654321
```

---

long long Modifier

```c
#include<stdio.h>

int main()
{
    long long int num = 9876543210123;

    printf("%lld",num);

    return 0;
}
```

Output

```
9876543210123
```

---

9. Type Casting

Overview

Type casting is used to convert one data type into another data type.

Syntax

```
(type)expression;
```

Example Program

```c
#include<stdio.h>

int main()
{
    float x = (float)5 / 2;

    printf("%f",x);

    return 0;
}
```

Output

```
2.500000
```

Additional Practice Programs

Nested printf()

```c
#include<stdio.h>

int main()
{
    printf("%d",printf("%d",printf("googleclassroom")));

    return 0;
}
```

Output

```
googleclassroom1517
```

---

printf() Return Value

```c
#include<stdio.h>

int main()
{
    int x = printf("hi");

    printf("%d",x + 2);

    return 0;
}
```

Output

```
hi4
```

Arithmetic Expression Evaluation

```c
#include<stdio.h>

int main()
{
    printf("%d",12 * 2 / 2 - 6 * 3 + 10);

    return 0;
}
```

Output

```
4
```

Nested Arithmetic printf()

```c
#include<stdio.h>

int main()
{
    printf("%d",printf("%d",12 + (6 / 2) * 5 / 2));

    return 0;
}
```

Output

```
192
```